

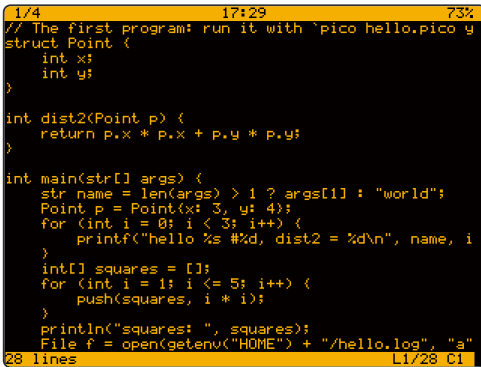
**Type along.** Every example here runs as it is: save it with `~/pico`, to read and change. `man pico` is the short reference on the machine.

|                             |                           |
|-----------------------------|---------------------------|
| 16                          | Programs to keep          |
| 15                          | Reference                 |
| 14                          | When it goes wrong        |
| 13                          | The internet              |
| 12                          | Digital rain              |
| 11                          | A game: snake             |
| 10                          | The terminal              |
| <b>PART 2: APPS</b>         |                           |
| 8                           | Text and structs          |
| 7                           | Functions and lists       |
| 6                           | Choosing: switch and enum |
| 5                           | Deciding and repeating    |
| 4                           | Values and variables      |
| 3                           | The first program         |
| <b>PART 1: THE LANGUAGE</b> |                           |
| <b>Contents</b>             |                           |

PART 1 OF 2 · THE LANGUAGE

# pico.

the language of PocketType



C-like and typed, **safe**: a mistake stops the program at its line, never the machine.

**Part 1** the language · **Part 2** apps: the terminal, games, the internet

1.0 beta 5

```
int main() {
    println("hello, pocket");
    return 0;
}

Write it with edit hello.pico, then Fn S to save and Fn Q to leave, and run it:

$ pico hello.pico
hello, pocket

pico compiles and runs in one go, piloc hello.pico makes a program of it, ./hello, that starts faster.

Every program has a main. What it returns is its exit status: 0 means all went well, main can have the words typed after the name:
```

## The first program

## Text and structs

```
str line = " the quick brown fox ";
str[] words = split(trim(line));
println(join(words, "-")); // the-quick-brown-fox
println(upper("hey"), find("hello", "ll"));
```

Also `substr`, `replace`, `contains`, `starts_with`, `repeat`, `chars`. `s[i]` is the byte at `i`, as a number.

### STRUCTS

```
struct Player { str name; int x; int lives; }

Player p = Player{name: "ada", x: 1, lives: 3};
p.x += 2;
println(p); // Player{name: "ada", x: 3, lives: 3}
```

Structs, lists and text are passed by reference: a function that gets a `Player` changes that same `Player`.

### FILES

```
File f = open("notes.txt", "w");
write(f, "a line\n");
close(f);
str all = read(open("notes.txt", "r"));
```

```
println("count is ", count);
printf("%s: %d at %.2f\n", name, count, price);

int count = 3;
float price = 2.50;
bool done = false;
str name = "Ada";
const int SIZE = 10;
// whole numbers
// with a point
// true or false
// text
// never changes

Every variable has a type, and says it:

PRINTING

TURNING ONE INTO ANOTHER

Nothing changes type by itself (except an int where a float is wanted):

str(42) "42"
to_int("17") 17, or 0 if it is not a number
int(3.9) 3
"n=" + n + joins text, numbers too
SUMS

+ - * / % as usual; x += 2, x++; 7 / 2 is 3.5, 7.0 / 2 is 3.5.
random(6) is 0 to 5, sqrt, pow, sin, abs, min, max.
```

## Values and variables

## Functions and lists

```
int square(int n) {
    return n * n;
}

void greet(str who, int times) {
    for (int i = 0; i < times; i++)
        println("hi ", who);
}
```

A function can be used above where it is written. One that returns something must return on every path: the compiler checks.

### LISTS (ARRAYS)

```
int[] scores = [12, 7, 30];
push(scores, 18); // [12, 7, 30, 18]
sort(scores); // [7, 12, 18, 30]
int best = scores[len(scores) - 1];
int last = pop(scores); // takes it off
int[][] grid = [[1, 2], [3, 4]];
```

Also `insert`, `remove_at`, `slice`, `reverse`, `find`. A wrong index stops the program and says where.

```
if (count > 5) {
    println("many");
} else if (count > 0) {
    println("some");
} else {
    println("none");
}

A condition is a bool: if (n != 0), never if (n). && and, || or, ! not.

while (lives > 0) { ... }
for (int i = 0; i < 10; i++) { ... }

EACH ITEM OF A LIST

str[] fruit = ["apple", "fig", "kiwi"];
for (str f in fruit) println("I like " + f);

for ... in cannot go past the end. For a text's letters: for
break leaves a loop; continue goes on with the next time
round.
```

## Deciding and repeating

## Choosing: switch and enum

When one value decides between many things:

```
switch (key) {
case 'q', KEY_ESC:
    running = false;
case KEY_UP:
    y--;
default:
    beeps++;
}
```

Only the case that matches runs — nothing falls through into the next one, as it does in C. A case can list several values; they can be numbers, text or enum values. `default` comes last.

### ENUM

Names for a handful of choices:

```
enum Dir { UP, DOWN, LEFT, RIGHT }

Dir d = LEFT;
switch (d) {
case UP, DOWN: println("up or down");
default: println("sideways");
}
```

Enum values are ints: UP is 0, DOWN 1, and LEFT = 10 sets one.

Also KEY\_UP, KEY\_DOWN, KEY\_ESC; Enter comes as '\n'.

```
}
print("\e[25H");
raw_mode(false);
}
if (key == KEY_RIGHT) x++;
if (key == KEY_LEFT) x--;
if (key == 'q') break;
int key = readkey(100);
printf("\e[10;50H 0 ", x);
for(;;) {
    print("\e[2J\e[25T");
    raw_mode(true);
    int x = 10;
```

regardless.
gives KEY\_NONE if nothing came, so the game moves on
raw\_mode(true). readkey(100) waits at most 100 ms and
A game needs each key the moment it's pressed:

### KEYS, AT ONCE

```
\e[31m \e[32m red, green text (30 to 37)
\e[1m \e[0m bold; back to normal
\e[2J clear the screen
\e[5;10H go to row 5, column 10
\e[25;10H hide, show the cursor
```

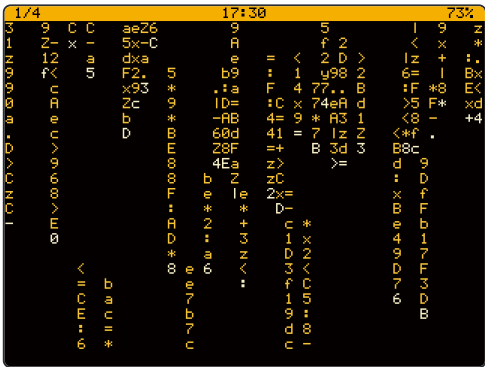
which are plain text:
term\_rows(). Colours and the cursor are escape codes.
The screen is 53 columns by 23 rows (term\_cols()),

## The terminal

### PART 2 OF 2 · APPS

# pico.

the language of PocketType



Part 2 colours and keys, a game loop, digital rain, the internet, commands, mistakes, and a reference on the back. All the programs are in ~/pico.

1.0 beta 5

```
}
}
default: return now;
...
case KEY_UP, 'w': return UP;
switch (key) {
    Dir steer(int key, Dir now) {
        while (lover) {
            int key = readkey(delay);
            dir = steer(key, dir);
            Point next = ahead(snake[0], dir);
            if (hits(next)) over = true;
            Insert(snake, 0, next); // the new head
            if (at(next, food)) grow();
            else erase(pop(snake)); // the tail
        }
    }
}
```

pico ~/pico/snake, pico: the whole game is a loop that reads a key, moves, and draws only what changed. In outline:



## A game: snake

## Programs to keep

In ~/pico on the machine, to run, read and take apart:

- matrix.pico digital rain, falling glyphs
- snake.pico the game: arrows or WASD
- weather.pico pico weather.pico Tokyo
- life.pico Conway's game of life
- mandel.pico a Mandelbrot zoom, in colour
- todo.pico a to-do list kept in a file
- guess.pico guess the number
- wc.pico counts lines, words, bytes
- hello.pico structs, arrays and a file
- benchmark.pico how fast the machine is

### IDEAS TO TRY

A clock in big digits. A dice roller. Flash cards from a file in ~/notes. The day's news headlines from a JSON feed. A pomodoro timer that runs beep with run(). A maze. A calculator.

### WHERE NEXT

man pico on the machine; on the PC, docs/PICO-TUTORIAL.md to learn and docs/LANGUAGE.md for all of it: every built-in, the limits, how the compiler works.

Run a program by its name. picoc snake.pico, put snake in ~/bin, and snake is a command.

```
}
}
default: return now;
...
case KEY_UP, 'w': return UP;
switch (key) {
    Dir steer(int key, Dir now) {
        while (lover) {
            int key = readkey(delay);
            dir = steer(key, dir);
            Point next = ahead(snake[0], dir);
            if (hits(next)) over = true;
            Insert(snake, 0, next); // the new head
            if (at(next, food)) grow();
            else erase(pop(snake)); // the tail
        }
    }
}
```

way it goes:
Steering is a switch over the keys, and an enum names the
The trick of any fast screen: build the frame as one text, write it once, and write only the cells that change.

```
}
}
print(frame);
if (readkey(50) == 'q') break;
}
drops[x] = new_drop();
if (d.y - d.len >= rows)
    frame += at(d.y - d.len, x) + " ";
frame += at(d.y - 1, x) + HEAD + glyph();
d.y++;
Drop d = drops[x];
for (int x = 0; x < cols; x++) {
    str frame = "";
    struct Drop { int y; int len; int speed; }
```

outline:
pico ~/pico/matrix.pico: a drop for each column falls, a bright head writing new glyphs, a trail fading behind it. In

## Digital rain

## Reference

```
int float bool str T[] File void
struct S { int a; } enum E { A, B }
const int N = 3; int[] a = [1, 2];
if else while for (;;) for (T x in a)
switch case default break continue return
?: && || ! + - * / % == != < > =
```

- print println printf format
- len of text, list
- push pop insert remove\_at
- sort reverse slice find
- split join trim chars
- subst replace repeat
- upper lower contains
- starts\_with ends\_with
- to\_int to\_float str
- chr ord codes
- abs min max sqrt pow
- sin cos tan atan2 hypot
- floor ceil round exp ln
- random seed PI
- open close read write
- readline exists is\_dir
- listdir mkdir remove
- rename stdin stdout
- run output getenv exit
- http\_get json\_get
- sleep\_ms uptime\_ms
- time date the clock
- raw\_mode readkey
- term\_cols term\_rows
- assert checks
- KEY\_UP ... KEY\_ESC

pico f.pico runs, picoc f.pico compiles, man pico reminds.

ANY COMMAND
output("date") runs a command and gives back what it
printed; run("play", "x.mp3") runs one and lets it print.

```
$ pico ~/pico/weather.pico Roma
25.4 C, clear
humidity 50%, wind 11.9 km/h
```

counts the list.
A path is names and numbers joined by dots: for {"list": [{"name": "a"}, {"name": "b"}]}

```
str j = http_get("https://api.open-meteo.com/?v1/forecast?latitude=41.9&longitude=12.5"
+ "&current_temperature_2m");
println(json_get(j, "current_temperature_2m"));
```

With Wi-Fi on, http\_get(url) fetches what an address answers, as text ("") if it fails). Web APIs answer in JSON, and json\_get(text, path) picks one value out of it.

## The internet

## When it goes wrong

The compiler says where, and often what to do:

```
hello.pico:4:13: error: expected int, got str;
parse it with to_int(s)
    int n = "12";
           ^
```

A mistake while it runs stops the program with the line, and how it got there; nothing else is harmed:

```
list.pico:6: runtime error: index 9 out of
range (length 3)
    in main (list.pico:6)
```

### FINDING MISTAKES

- println(x) show a value; lists and structs too
- assert(ok, "why") stop at once if something is wrong
- Fn C stop a program that runs away
- picoc -d x.pico the compiled instructions

### TIPS

Start from a program in ~/pico and change it. Keep functions short. Test on the PC too: make -C lang builds the same pico there. docs/PICO-TUTORIAL.md and docs/LANGUAGE.md have the rest.